Pensamento Computacional
INF05008

Aula 9 – Definições Locais e Recursão e Listas II

Prof. Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul

Instituto de Informática

Departamento de Informática Teórica

INSTITUTO DE INFORMÁTICA UFRGS

Definições Locais

Linguagem

- O objetivo de uma linguagem é transmitir informação.

- Para se construir uma linguagem são necessários:
  - Símbolos
  - Sintaxe
  - Semântica

Símbolos

- Quando escrevemos um texto, usamos os símbolos permitidos pela linguagem que estamos utilizando…

Sintaxe

- Para escrever as frases, seguimos as regras da sintaxe da linguagem, que dizem como devem ser formadas as sentenças.

- Em linguagem natural, as frases tendem a ter muitas palavras (não se concentrando estritamente na essência a ser transmitida).

- Linguagens de programação tendem a ser muito mais objetivas.

Semântica

- Semântica é o significado das palavras ou símbolos de uma linguagem.

casa 

saudade 

escolha 

- Novos símbolos podem ser criados, e sua semântica definida:
  - “Sepá”
  - Internet
  - MESA = circle(100, “solid”, “lightgrey”)

Execução de Programas

- Para entender um texto, precisamos conhecer a semântica de todas palavras usadas no texto.

- Para executar um programa, o computador precisa entender a semântica de todas palavras usadas no programa.

P. Como o computador faz isso?

- O computador guarda uma tabela com as palavras conhecidas e seu significado (como um dicionário).
- Essa lista pode ser modificada durante a execução de um programa…

Tabela de Nomes

Nome	Tipo	Valor (significado)
false	constante	Valor lógico “falso”
num-sqr	função	Função que eleva um número ao quadrado
circle	função	Função que gera a imagem de um círculo

- Em Pyret, estes nomes são pré-definidos, sua semântica está embutida no processador da linguagem.
- Mas podemos definir novos nomes!

Definindo Nomes

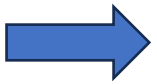
- Novos nomes podem ser criados, e sua semântica definida:

```
PI = 3.14
MESA = circle(200, "solid", "lightgray")
fun media(a :: Number, b :: Number) -> Number:
  doc: "Dado dois números, retorna a média entre eles."
  (a + b) / 2
end
```

Nome	Tipo	Valor (significado)
PI	constante	3.14
MESA	constante	Imagem de um círculo cinza de raio 200

media	função	`fun media(a, b):`
		`  (a + b) / 2`
		`end`

Exemplo

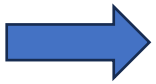


```
fun media(a :: Number, b :: Number) -> Number:
  (a + b) / 2
end

media(2, 4)
```

Nome	Tipo	Valor (significado)
media	função	fun media(a, b): (a + b) / 2 end

Exemplo



```
fun media(a :: Number, b :: Number) -> Number:
  (a + b) / 2
end

media(2, 4)
```

Nome	Tipo	Valor (significado)
media	função	fun media(a, b): (a + b) / 2 end
a	constante	2
b	constante	4

Exemplo

```
fun media(a :: Number, b :: Number) -> Number:
  (a + b) / 2
end

media(2, 4)
>> 3
```



Nome	Tipo	Valor (significado)

media	função	`fun media(a, b):`
		`  (a + b) / 2`
		`end`

- As constantes a e b só existem durante a execução da função media!

Definições Locais

Muitas vezes, dentro do escopo de uma definição, podemos dar nomes a algumas expressões, com o objetivo de:

- **Tornar a definição mais organizada**
- **Legível**
- **Evitar a repetição de cálculos**.

Exemplo

Como calcular as raízes de uma equação de segundo grau?

1. Calcula-se o discriminante $\Delta = b^2 - 4ac$
2. Temos 3 casos:
   - se $\Delta < 0$, não há raízes
   - se $\Delta = 0$, há uma raiz, dada por $x = \frac{-b}{2a}$
   - se $\Delta > 0$, há duas raízes: $x' = \frac{-b + \sqrt{\Delta}}{2a}$ $x'' = \frac{-b - \sqrt{\Delta}}{2a}$

P. Por que usamos o Δ ? Qual o seu papel?

Exemplo

```
fun raizes(a :: Number, b :: Number, c :: Number) -> List<Number>:
  doc: "Dado os coeficientes de uma equação quadrática, retorna as suas raízes."
  ask:
  | (num-sqr(b) - ((4 * a) * c)) < 0 then: empty # lista vazia
  | (num-sqr(b) - ((4 * a) * c)) == 0 then: [list: ((0 - b) / (2 * a))]
  | otherwise:
    [list:  ((0 - b) + num-sqrt(num-sqr(b) - ((4 * a) * c))) / (2 * a),
            ((0 - b) - num-sqrt(num-sqr(b) - ((4 * a) * c))) / (2 * a)]
  end
end
```

- A expressão para computar o delta é utilizada várias vezes na definição da função.

Exemplo

```
fun raizes(a :: Number, b :: Number, c :: Number) -> List<Number>:
  doc: "Dado os coeficientes de uma equação quadrática, retorna as suas raízes."
  delta = num-sqr(b) - ((4 * a) * c)

  ask:
  # Se delta é menor que zero, devolver a lista vazia (não tem raízes)
  | delta < 0 then: empty

  # Se delta é igual a zero, devolver uma lista com a raíz única
  | delta == 0 then: [list: ((0 - b) / (2 * a))]

  # Se delta é maior que zero, devolver a lista com as duas raízes
  | otherwise:
    [list: ((0 - b) + num-sqrt(delta)) / (2 * a),
           ((0 - b) - num-sqrt(delta)) / (2 * a)]
  end
end
```

Exemplo

```
fun raizes(a :: Number, b :: Number, c :: Number) -> List<Number>:
  doc: "Dado os coeficientes de uma equação quadrática, retorna as suas raízes."
  delta = num-sqr(b) - ((4 * a) * c)
  ask:
  # Se delta é menor que zero, devolver a lista vazia (não tem raízes)
  | delta < 0 then: empty

  # Se delta é igual a zero, devolver uma lista com a raíz única
  | delta == 0 then: [list: ((0 - b) / (2 * a))]

  # Se delta é maior que zero, devolver a lista com as duas raízes
  | otherwise:
    [list: ((0 - b) + num-sqrt(delta)) / (2 * a),
           ((0 - b) - num-sqrt(delta)) / (2 * a)]
  end
end
```

definição local

expressão

Exemplo

```
fun raizes(a :: Number, b :: Number, c :: Number) -> List<Number>:
  doc: "Dado os coeficientes de uma equação quadrática, retorna as suas raízes."
  delta = num-sqr(b) - ((4 * a) * c)
  menos-b = (0 - b)

  ask:
  # Se delta é menor que zero, devolver a lista vazia (não tem raízes)
  | delta < 0 then: empty

  # Se delta é igual a zero, devolver uma lista com a raíz única
  | delta == 0 then: [list: (menos-b / (2 * a))]

  # Se delta é maior que zero, devolver a lista com as duas raízes
  | otherwise:
    [list: (menos-b + num-sqrt(delta)) / (2 * a),
           (menos-b - num-sqrt(delta)) / (2 * a)]
  end
end
```

definições locais

expressão

Exemplo

```
fun cria-carta(nome :: String, tipo :: String) -> Image:
    doc: "Dado o nome do Pokemon e o tipo da carta, devolve uma imagem com a carta montada."

    # Coloca a borda sobre a carta
    overlay-align("center", "center",
        # Coloca o nome no topo da carta
        overlay-align("middle", "top",
            text(nome, 20, "black"),
            # Coloca o tipo na parte inferior da carta
            overlay-align("middle", "bottom",
                text(tipo, 20, "black"),
                # Coloca o fundo da carta
                overlay(
                    # Coloca a imagem do pokemon sobre o círculo branco
                    overlay(
                        seleciona-imagem-pokemon(nome),
                        circle(60, "solid", "white")),
                    seleciona-fundo(tipo)))),
    BORDA)
end
```

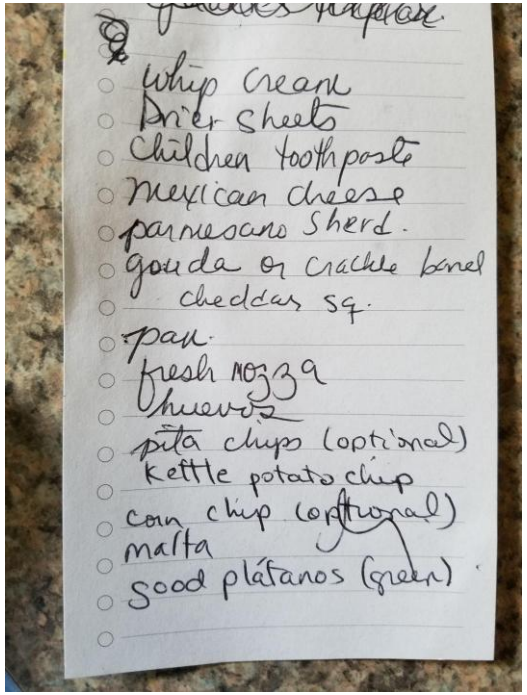
Exemplo

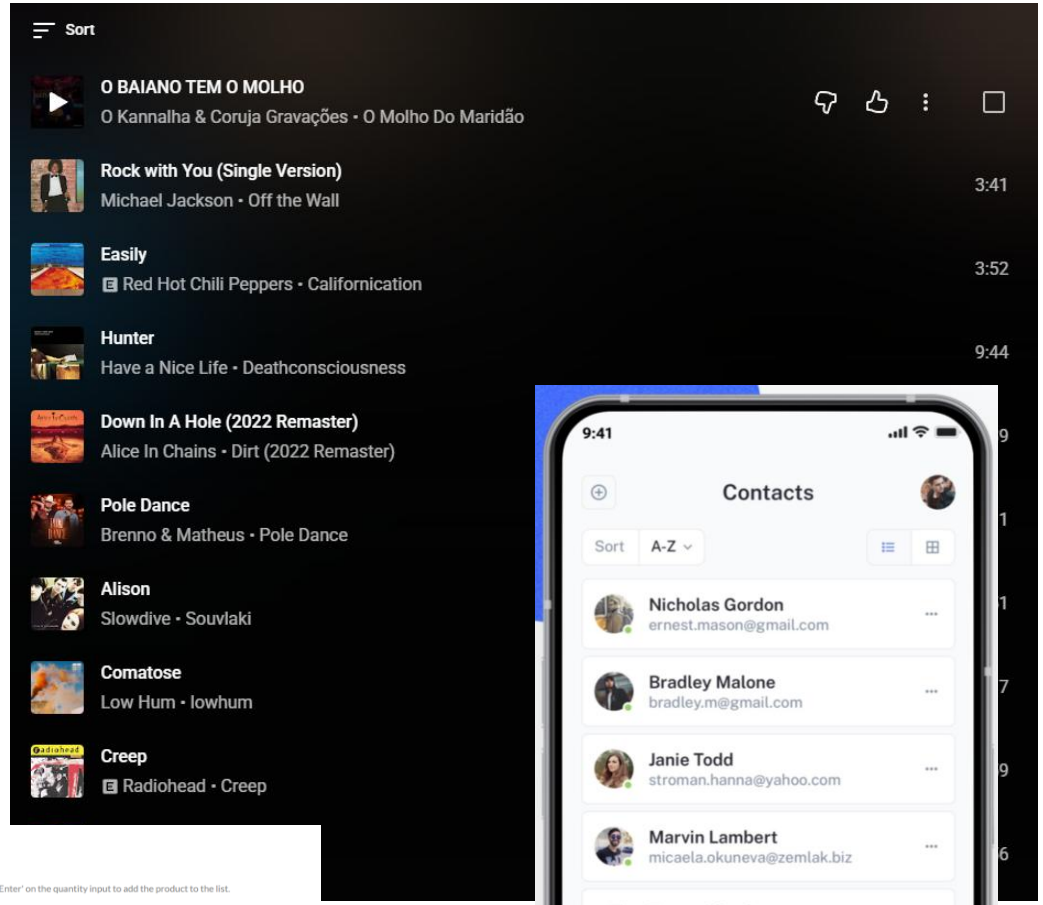
```
fun cria-carta-v2(nome :: String, tipo :: String) -> Image:
    doc: "Dado o nome do Pokemon e o tipo da carta, devolve uma imagem com a carta montada."

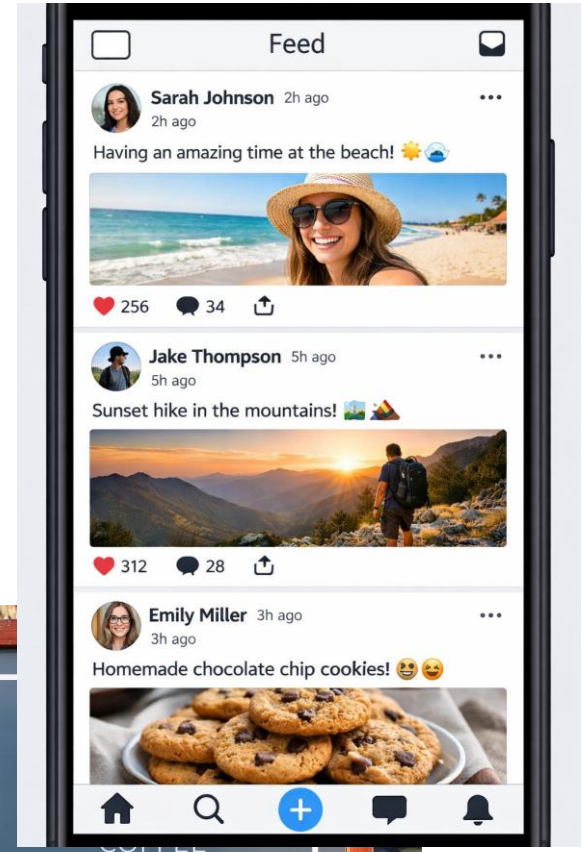
    # Definições Locais:
    # Gera imagem do pokemon sobre o círculo
    pokemon-sobre-circulo = overlay(seleciona-imagem-pokemon(nome), circle(60, "solid", "white"))
    # Gera fundo com o pokemon
    fundo-com-pokemon = overlay(pokemon-sobre-circulo, seleciona-fundo(tipo))
    # Gera carta com o tipo
    carta-com-tipo = overlay-align("middle", "bottom", text(tipo, 20, "black"), fundo-com-pokemon)
    # Gera carta com o nome
    carta-com-nome = overlay-align("middle", "top", text(nome, 20, "black"), carta-com-tipo)
    # Gera carta final com a borda
    carta-final = overlay-align("center", "center", carta-com-nome, BORDA)

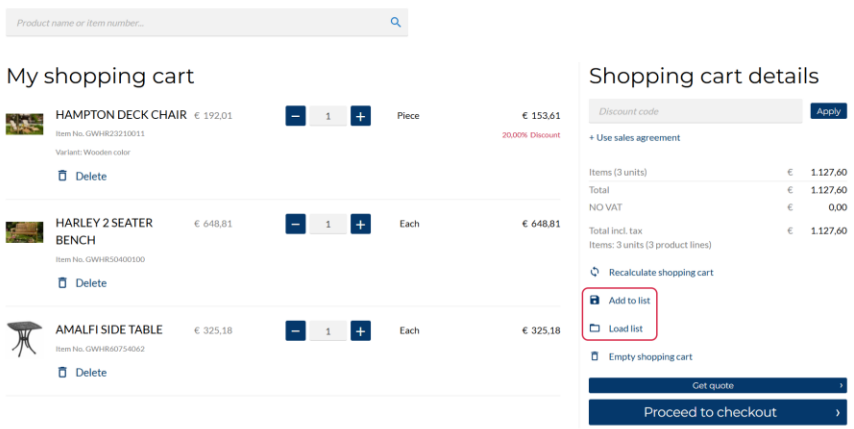

    # Expressão:
    carta-final
end
```

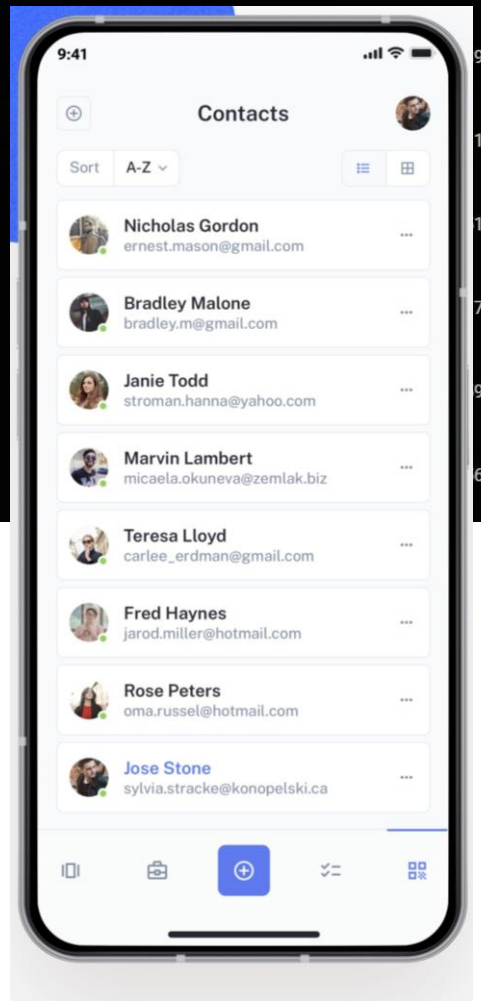
Listas

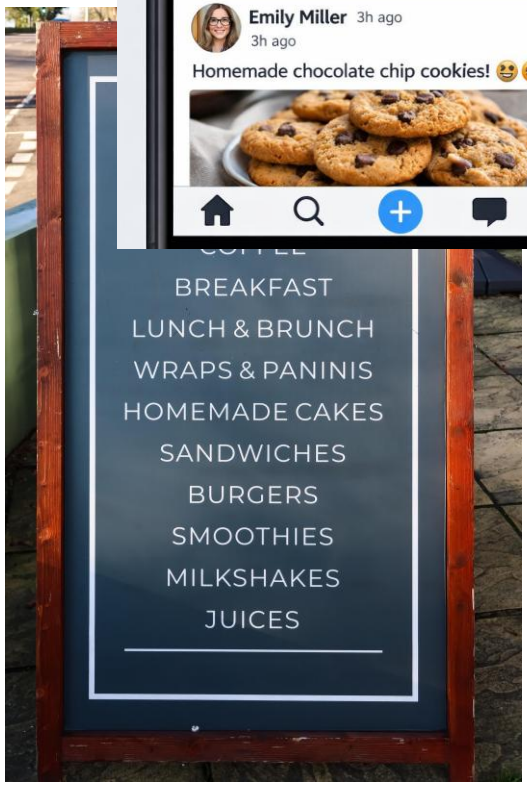

whip cream
Brier sheets
Children toothpaste
mexican cheese
parmesano sherd.
gouda or cracker barrel
cheddar sq.
pan.
fresh mozza
chuevos
pita chips (optional)
kettle potato chip
corn chip (optional)
malta
sood plátanos (green)


Sort
O BAIANO TEM O MOLHO
O Kannalha & Coruja Gravações • O Molho Do Maridão
Rock with You (Single Version)
Michael Jackson • Off the Wall
3:41
Easily
Red Hot Chili Peppers • Californication
3:52
Hunter
Have a Nice Life • Deathconsciousness
9:44
Down In A Hole (2022 Remaster)
Alice In Chains • Dirt (2022 Remaster)
Pole Dance
Brenno & Matheus • Pole Dance
Alison
Slowdive • Souvlaki
Comatose
Low Hum • lowhum
Creep
Radiohead • Creep


Feed
Sarah Johnson 2h ago
2h ago
Having an amazing time at the beach!
256
34
Jake Thompson 5h ago
5h ago
Sunset hike in the mountains!
312
28
Emily Miller 3h ago
3h ago
Homemade chocolate chip cookies!


Shopping cart
Enter an item number and press 'Enter' to load the product information and variants. Tab to select variants and set quantity. Press 'Enter' on the quantity input to add the product to the list.
Product name or item number...
My shopping cart
HAMPTON DECK CHAIR € 192,01
Item No. GWHR23210011
Variant: Wooden color
Delete
Piece
€ 153,61
20,00% Discount
HARLEY 2 SEATER BENCH € 648,81
Item No. GWHR50400100
Delete
Each
€ 648,81
AMALFI SIDE TABLE € 325,18
Item No. GWHR60754062
Delete
Each
€ 325,18
Shopping cart details
Discount code
Apply
+ Use sales agreement
Items (3 units) € 1.127,60
Total € 1.127,60
NO VAT € 0,00
Total incl. tax € 1.127,60
Items: 3 units (3 product lines)
Recalculate shopping cart
Add to list
Load list
Empty shopping cart
Get quote
Proceed to checkout


9:41
Contacts
Sort
A-Z
Nicholas Gordon
ernest.mason@gmail.com
Bradley Malone
bradley.m@gmail.com
Janie Todd
stroman.hanna@yahoo.com
Marvin Lambert
micaela.okuneva@zemlak.biz
Teresa Lloyd
carlee_erdman@gmail.com
Fred Haynes
jarod.miller@hotmail.com
Rose Peters
oma.russel@hotmail.com
Jose Stone
sylvia.stracke@konopelski.ca


BREAKFAST
LUNCH & BRUNCH
WRAPS & PANINIS
HOMEMADE CAKES
SANDWICHES
BURGERS
SMOOTHIES
MILKSHAKES
JUICES

Lista

- Uma lista é uma estrutura fundamental na computação usada para agrupar elementos.

- Possui tamanho indeterminado;
- Não se pode acessar qualquer elemento diretamente;
  - É necessário percorrer à partir do primeiro elemento.

- Ordem: Os elementos têm ordem definida (primeiro, segundo, último).
- Tipagem Homogênea: Todos os elementos de uma mesma lista são do mesmo tipo.
- Dado Anônimo: A lista em si não diz "o que" ela representa (ex: idades? preços?). Você, como programador(a), é quem dá o contexto.

Indução e Recursão



Recursão

Ideia geral:

- Se o problema for trivial, resolva-o; (fim)
- Caso contrário:
  - Quebre/decomponha o problema
  - Assuma que os problemas menores estão solucionados
  - Combine as soluções

Conjunto Naturais

O conjunto dos números naturais pode ser definido pelas 3 regras a seguir:

R1. $0 \in \mathbb{N}$ ***(caso base)***

R2. Se $n \in \mathbb{N}$, o sucessor de n pertence a $\mathbb{N}$ ***(passo)***

R3. Somente elementos obtidos a partir das regras R1 e R2 pertencem a $\mathbb{N}$ ***(exclusão)***

Conjunto NumList

O conjunto das Listas de Números pode ser definido pelas 3 regras a seguir:

R1. nl-empty ∈ NumList *(caso base)*

R2. Se n ∈ Number e l ∈ NumList, então nl-link(n, l) ∈ NumList *(passo)*

R3. Somente elementos obtidos a partir das regras R1 e R2 pertencem a $\mathbb{N}$ *(exclusão)*

```
data NumList:
    | nl-empty  # lista vazia
    | nl-link(first :: Number, rest :: NumList)
    # first é o primeiro número da lista, rest é o restante da lista
end
```

Dado vs. Programa

```
data NumList:
    | nl-empty

    | nl-link(
       first :: Number,
       rest :: NumList)
end
```

```
cases (NumList) nl:
   | nl-empty => <expressão>

   | nl-link(f, r) => <expressão>
end
```

Expressão **cases**

Em Pyret, distinguimos as variantes de um dado condicional com a expressão **cases**:

```
cases (NumList) nl:
   | nl-empty => <expressão>
   | nl-link(f, r) => <expressão>
end
```

- `nl` é uma expressão que avalia para um valor do tipo `NumList`
- `f` e `r` são nomes dados para os atributos first e rest
- O lado direito de todo "`=>`" é uma expressão

Listas - Sintaxe

- Podemos usar a notação `[list: a, b, c]` para criar valores do tipo List.

- Exemplos práticos:

```
[list: 10, -1, 5, 2.3]  # (Lista de Números)

[list: "maçã", "banana", "uva"]  # (Lista de Strings)

shopping = [list: "pão", "leite"]  # Dando nome a uma lista

[list: ] ou empty  # (Uma lista vazia. Pode parecer inútil agora, mas será muito importante depois!)
```

Criando Listas

A expressão [list:] é apenas uma sintaxe alternativa para empty e link:

```
check:
    empty is [list: ]

    link(1, link(2, link(3, empty))) is [list: 1, 2, 3]

    link("a", link("b", link("c", empty))) is [list: "a", "b", "c"]
end
```

- Para criar funções que operam sobre listas, precisaremos decompor listas usando **empty** e **link**!

Tipo Lista Pré-Definido

- Em Pyret, o tipo List já é pré-definido:

```
data List<T>:

  | empty       # Caso Base

  | link(first :: T, rest :: List<T>)  # Passo

end
```

- Na definição acima, <T> indica que podemos criar listas de qualquer tipo T!

```
l1 :: List<String> = [list: "a", "b", "c"]

l2 :: List<Number> = [list: 1, 2, 3, 4, 5]

l3 :: List<Boolean> = [list: true, false, true]
```

Recursão Estrutural

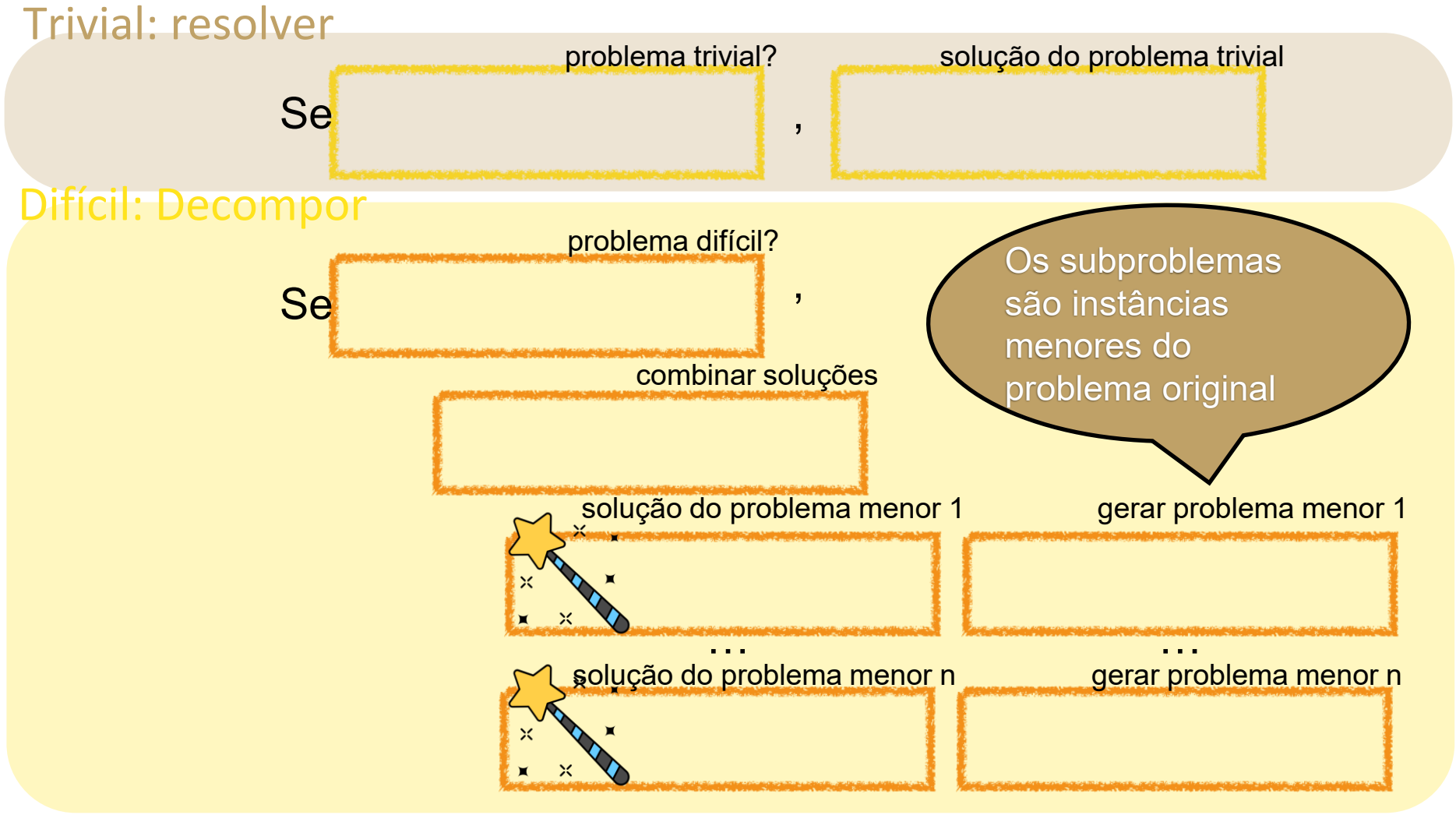
Expressão **cases**

Em Pyret, distinguimos as variantes de um dado condicional com a expressão **cases**:

```
cases (Tipo) e:
  | variante-1(x, y) => <expressão>
  | ...
  | variante-n(x, y, z) => <expressão>
end
```

- e é uma expressão que avalia para um valor do tipo Tipo
- Primeiro, e é avaliado e é checado se e é do tipo Tipo. Se não for, erro de execução.
- Se é Tipo, checa qual variante, e retorna a expressão a direita do => correspondente .

Modelo da solução



Template para Recursão de Listas

```
fun num-list-fun(nl :: NumList) -> ???:
  cases (NumList) nl:
    | nl-empty => ...

    | nl-link(first, rest) =>
      ... first ...
      ... num-list-fun(rest) ...
  end
end
```

- Ler: https://dcic-world.org/2025-08-27/recursive-data.html#(part._.The_.Design_.Recipe)

Contem 7

Trivial: resolver

Se **lista vazia** (problema trivial?), **falso** (solução do problema trivial)

Difícil: Decompor

Se **link(first, rest)** (problema difícil?),

combinar soluções: **ou**

solução do problema menor	gerar problema menor
Verificar se == 7	first
solução do problema menor 2	gerar problema menor
Verificar se contem 7	rest

Contem 7

```
fun contem-7(nl :: NumList) -> Boolean:
  doc: "Dado uma lista de números, retorna true se ela contém o número 7, e false caso contrário."
  cases (NumList) nl:
    # Se a lista for vazia, ela não contém o número 7
    | nl-empty => false
    # Se a lista não for vazia:
    | nl-link(first, rest) =>
        # Se o primeiro número da lista for 7, então a retorna true
        if first == 7:
            true
        # Se não, verificar se o resto da lista contem o número 7
        else:
            contem-7(rest)
        end
  end
where:
  contem-7(nl-empty) is false
  contem-7(nl-link(1, nl-link(2, nl-link(3, nl-empty)))) is false
  contem-7(nl-link(1, nl-link(7, nl-empty))) is true
end
```

Recursão Estrutural para Transformar Listas

Transformando Listas

- Na função contem-7, processamos uma NumList sem modificá-la.
- Vamos fazer uma função que aplica uma transformação em cada elemento da lista!

dobro

Obj: Dada uma lista de números, retorna uma lista com todos os números dobrados.

Entrada: Uma lista de números.

Saída: Uma lista de números com todos os números multiplicados por 2.

dobro

Trivial: resolver

Se lista vazia (problema trivial?), lista vazia (solução do problema trivial)

Difícil: Decompor

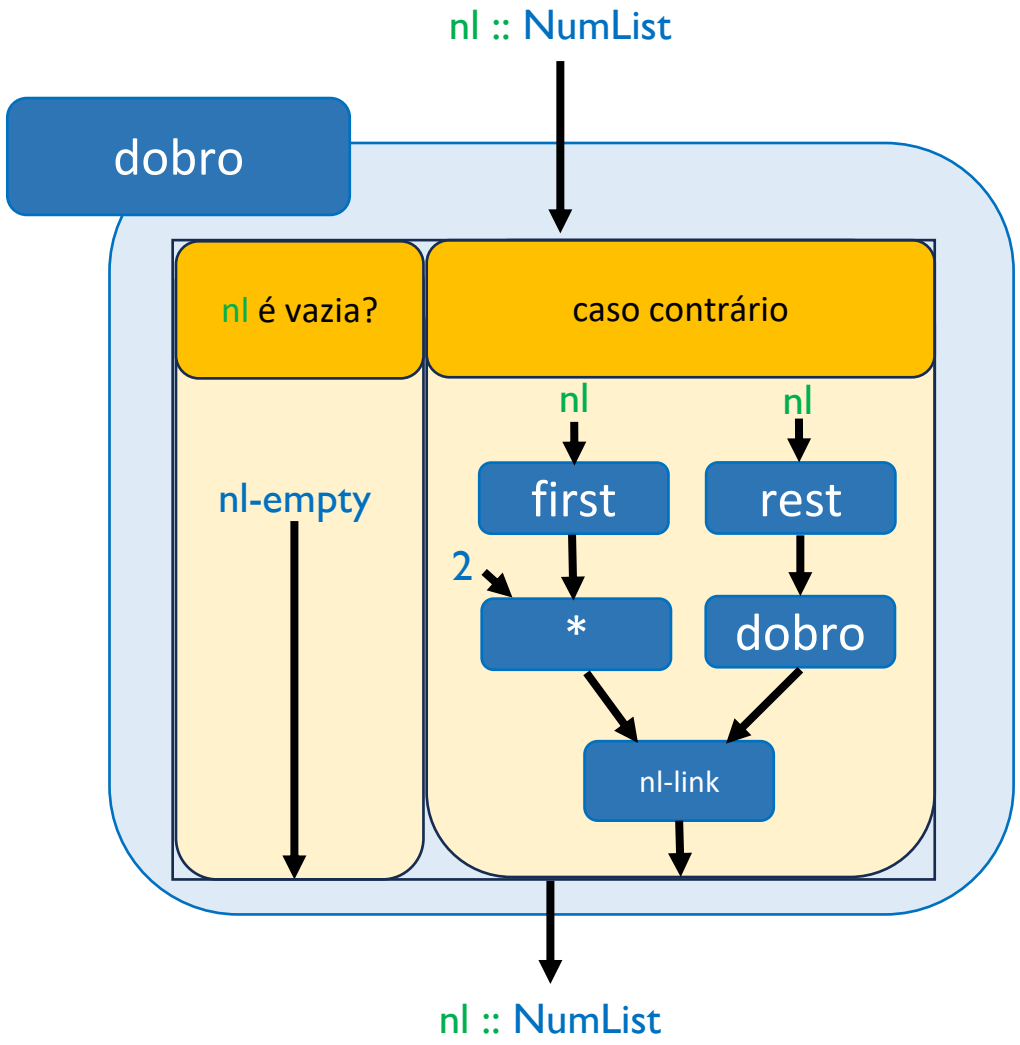
Se link(first, rest) (problema difícil?),

combinar soluções: link

solução do problema menor: * 2 — gerar problema menor: first

solução do problema menor 2: dobro — gerar problema menor: rest

Dobro




```
fun dobro(nl :: NumList) -> NumList:
    doc: "Dado uma lista de números, devolve uma nova
lista com o dobro de cada número."
    cases (NumList) nl:
      # Se a lista é vazia
      | nl-empty =>
          nl-empty
      # Se a lista é não vazia
      | nl-link(first, rest) =>
          nl-link(              # “Linka”
                2 * first,     # o dobro do primeiro
                dobro(rest)) # resto da lista dobrada
    end
where:
    dobro(nl-empty) is nl-empty
    dobro(nl-link(1, nl-empty)) is nl-link(2, nl-empty)
    dobro(nl-link(2, nl-link(3, nl-empty))) is nl-link(4,
nl-link(6, nl-empty))
end
```

Recursão Estrutural para Filtrar Listas

Filtrando Listas

- E se quisermos filtrar/remover determinados elementos de uma lista?
- Vamos fazer uma função que filtra número negativos de uma lista de números.

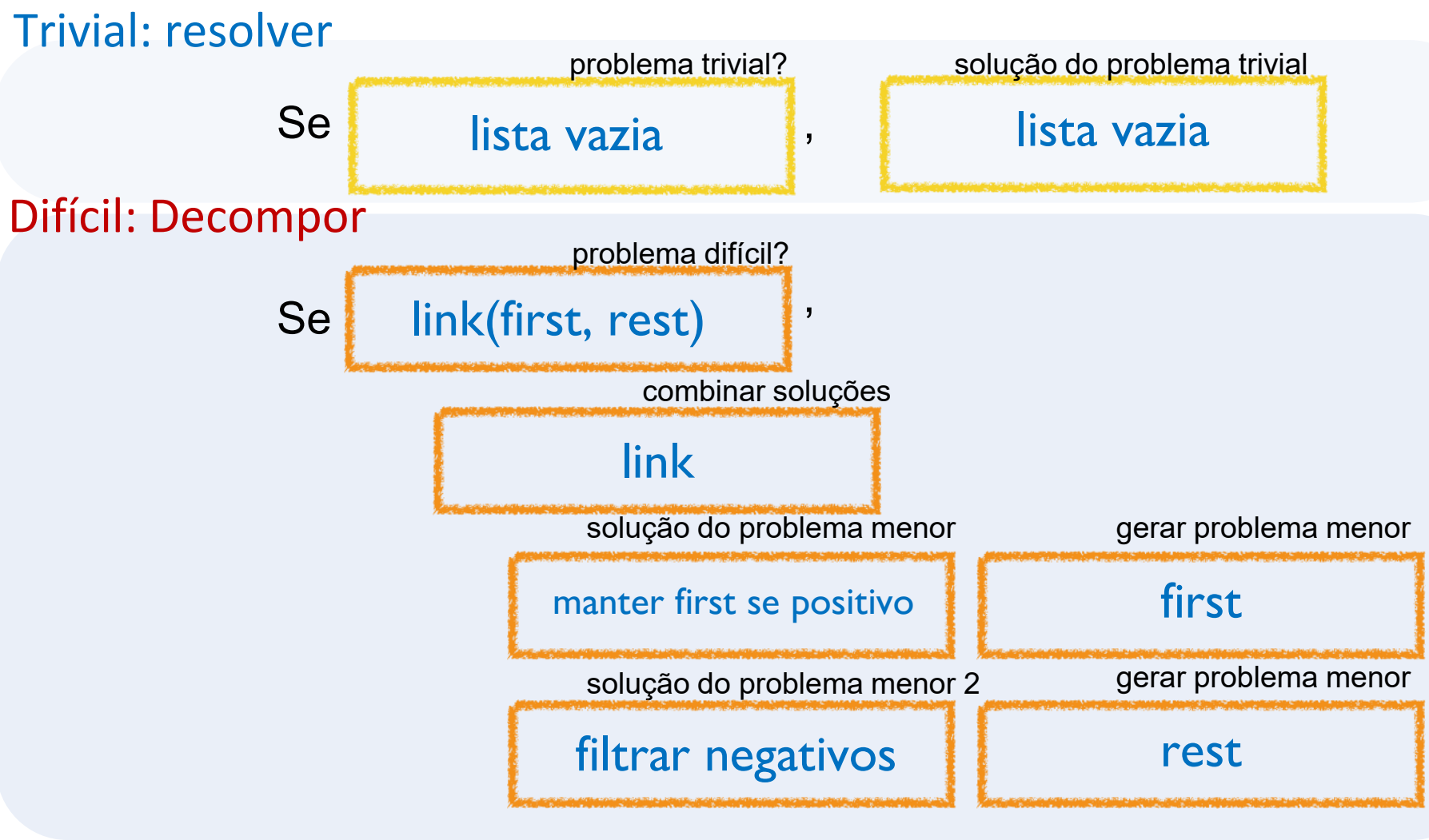
Filtra Negativos

Obj: Dada uma lista de números, retorna a lista apenas com números positivos.

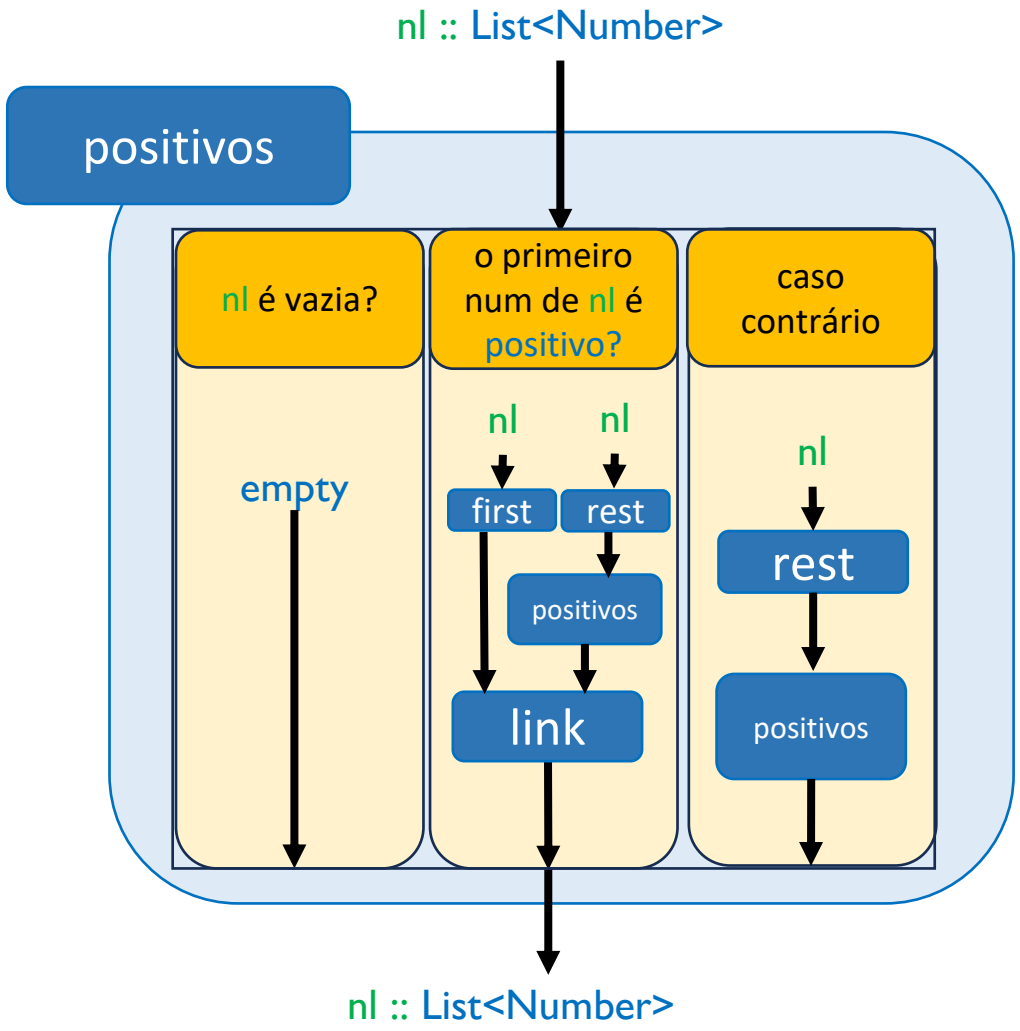
Entrada: Uma lista de números.

Saída: Uma lista de números apenas com os números positivos da lista de entrada.

Filtra Negativos



Positivos




```
fun positivos(nl :: List<Number>) -> List<Number>:
    doc: "Dado uma lista de números, devolve uma nova
lista contendo apenas os números positivos."
    cases (List<Number>) nl:
      | empty => empty
      | link(f, r) =>
          ask:
              | f > 0 then:
                  link(f, positivos(r))
              | otherwise:
                  positivos(r)
          end
    end
where:
    positivos(empty) is empty
    positivos(link(1, empty)) is link(1, empty)
    positivos(link(-1, link(2, empty))) is link(2,
empty)
end
```

Filtrando Listas

- Vamos remover duplicatas de uma lista!

Remover Duplicatas

Obj: Dada uma lista de números, retorna a lista mantendo apenas uma aparição de cada elemento repetido (o mais para o final da lista).

Entrada: Uma lista de números.

Saída: Uma lista de números sem duplicatas.

Exemplos

```
examples:
  uniq([list: 1, 2, 1, 3, 1, 2, 4, 1]) is [list: 3, 2, 4, 1]
  uniq([list:    2, 1, 3, 1, 2, 4, 1]) is [list: 3, 2, 4, 1]
  uniq([list:       1, 3, 1, 2, 4, 1]) is [list: 3, 2, 4, 1]
  uniq([list:          3, 1, 2, 4, 1]) is [list: 3, 2, 4, 1]
  uniq([list:             1, 2, 4, 1]) is [list:    2, 4, 1]
  uniq([list:                2, 4, 1]) is [list:    2, 4, 1]
  uniq([list:                   4, 1]) is [list:       4, 1]
  uniq([list:                      1]) is [list:          1]
  uniq([list:                       ]) is [list:           ]
end
```

```
fun contem(n :: Number, lst :: List<Number>) -> Boolean:
  doc: "Verifica se um número está presente em uma lista."
  cases (List<Number>) lst:
    | empty => false
    | link(f, rest) =>
      ask:
        | f == n then: true
        | otherwise: contem(n, rest)
      end
  end
end
```

```
fun uniq(l :: List<Number>) -> List<Number>:
  doc: "Recebe uma lista e devolve uma nova lista com os mesmos
elementos, mas sem repetições."
  cases (List<Number>) l:
    | empty => empty
    | link(f, r) =>
      ur = uniq(r)
      if contem(f, ur):
        ur
      else:
        link(f, ur)
      end
  end
end
```

Funcional vs. Imperativo

- Vamos comparar como escreveríamos uma função para somar todos os elementos de uma lista ou array no paradigma funcional e imperativo.

```
fun soma(lst :: List<Number>) -> Number:
  cases (List<Number>) lst:
    | empty => 0
    | link(first, rest) => first + soma(rest)
  end
end
```

```
int soma(int arr[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++) {
        total += arr[i];
    }
    return total;
}
```

Conceito	**Pyret (funcional)**	**C (imperativo)**
Estrutura	Baseada no dado	Baseada em controle
Iteração	Recursão	Loop (*for*)
Estado	Imutável	Mutável
Forma da solução	Declarativa	Procedural

